

**BHASKARACHARYA NATIONAL INSTITUTE FOR
SPACE APPLICATIONS AND GEO-INFORMATICS**
WEEKLY PROGRESS REPORT (15/06/2026 – 21/06/2026)

WEEK 5

Project Name	OSINT Collection from Social Media Network
PROJECT DESCRIPTION :	Continuation of the linkmap OSINT pipeline. Week 5 focused on finalising and stabilising the Nexus graph explorer with two major additions: a date-range handle filter with multi-select handle picker, and an automated PDF OSINT report generator that compiles per-handle intelligence profiles, cross-handle relationship analysis, and narrative inferences into a downloadable report. The week also fixed a critical SPA routing bug that was intercepting file downloads.
GROUP MEMBER :	Rishav Pal, Sayantan Mandal, Shivam Kumar
GROUP ID :	2026G268
GROUP GUIDE :	Dr. JHUMMARWALA ABDUL
GROUP COORDINATOR :	Dr. JHUMMARWALA ABDUL
COLLEGE GUIDE NAME :	Dr. SHWETA SAHARAN
COLLEGE GUIDE No. :	9509676076
COLLEGE GUIDE EMAIL. :	SHWETA.SAHARAN@GSV.AC.IN
COLLEGE NAME :	GATI SHAKTI VISHWAVIDYALAYA

WEEK 5 — DAILY WORK LOG

Day 1–2 | 15–16 June 2026 | Repository Consolidation & Production Readiness Review

The first two days were spent consolidating the entire project into a single version-controlled repository and conducting a full review of all prior fixes before beginning new feature work.

- Initialised git version control for the complete linkmap project with a structured .gitignore covering virtual environments, data directories, logs, and environment secrets.
- Committed the full production-ready codebase as a baseline: all 11 frontend pages, 10 backend route files, the scraper pipeline, Neo4j graph layer, and PostgreSQL storage layer — establishing a clean starting point for Week 5 feature additions.
- Reviewed and confirmed all bug fixes from prior weeks remained intact in the committed baseline: connection pool usage in postgres_writer.py, batch tweet writes in main.py, FastAPI lifespan context manager, asyncio.get_running_loop(), and the lxml-based ReDoS-safe HTML parser in scraper/parser.py.
- Audited rolling log output across all modules (auth, browser, scraper, parser, rate limiter, postgres_writer, transformer, cypher_loader) to confirm the logging infrastructure was capturing data correctly across repeated scrape and graph-build cycles.
- Planned the Week 5 feature scope: a date-range filterable handle picker on the Nexus page and a downloadable PDF intelligence report — both requested as natural extensions of the existing per-handle detail view built in Week 4.

Day 3 | 17 June 2026 | Date-Range Handle Picker — Backend & Filter Bar UI

Day 3 began the Nexus enhancement work by building the backend endpoint and frontend filter bar that lets a user narrow the handle set by a date range before generating a report.

- Built GET /api/nexus/handles-by-daterange in backend/routes/nexus.py — accepts optional start and end query parameters, parses flexible datetime-local string formats via a new parse_dt() helper supporting four date/time format variants, and falls back gracefully to "no filter" when either bound is omitted.
- Wrote the SQL query joining profiles to tweets with a dynamic WHERE clause built from the supplied date bounds, returning each matching handle with tweet_count_in_range, earliest_tweet, and latest_tweet — giving the picker enough context to show tweet volume per handle directly in the selection list.
- Designed and built the filter bar UI in frontend/nexus.html: a horizontal bar with "From" and "To" datetime-local inputs, a primary action button, a status text element, and an Export PDF button — all styled to match the existing dark Nexus canvas aesthetic.
- Built the handle-picker dropdown component — a checkbox list panel that opens beneath the filter bar showing every handle matching the date range, each row displaying the @handle, display name, formatted follower count (e.g. 1.2M), and the tweet count within the selected range. Includes Select All and Clear actions and a close control.
- Added .handle-picker CSS block (header, actions, list, item, selected state, checkbox, empty state) to keep the new component visually consistent with the dark graph panel while remaining legible against the existing component library.

Day 4 | 18 June 2026 | PDF Report Generator — Backend

Day 4 was the core feature day — building the full server-side PDF generation pipeline that turns a selection of handles and a date range into a structured intelligence report.

- Built POST /api/nexus/report accepting a ReportRequest body (handles list, optional start_dt, end_dt). Validates every handle against the standard regex pattern and silently drops any that fail validation rather than rejecting the whole request.
- Implemented per-handle data gathering: for each selected handle, queries profile data, date-bounded tweets, top 10 hashtags, top 10 mentions, top 5 shared links, top 10 topics, aggregate engagement statistics (total tweets, total likes, total retweets, average likes), and the single most-liked tweet — all scoped to the requested date range.
- Implemented cross-handle relationship detection across the selected set: shared_hashtags (hashtags used by 2 or more of the selected handles), cross_mentions (directional mention pairs where one selected handle mentions another), and shared_links (URLs shared by 2 or more selected handles) — the same coordination-signal logic used in earlier Intelligence page work, now scoped to an arbitrary user-chosen handle set.
- Built an automated inference engine that generates plain-English findings from the gathered data: identifies the most active handle by tweet volume, the most influential handle by total likes, flags the strongest hashtag coordination signal with the full list of handles involved, reports the count of cross-handle mentions detected, and surfaces topics common to multiple handles.

- Built the full ReportLab PDF document: a cover section with report title, period, handle list and generation timestamp; a per-handle profile section with a stats table (followers, following, tweets, likes, retweets, avg likes), side-by-side hashtag and mention tables, a topics line, and a most-liked-tweet quote block; a cross-handle relationships section with three tables (shared hashtags, cross mentions, shared links); and a final inferences section listing the auto-generated findings.
- Styled the PDF to match the platform's editorial design language — black header bars with off-white text, thin grid lines, Helvetica typography at consistent size tiers, and alternating row shading on every table for readability.
- Returned the generated PDF as a Response object (not StreamingResponse) using `buf.getvalue()` to avoid BytesIO garbage-collection issues in the async request context, with Content-Disposition and Content-Length headers set for proper browser download handling.

WEEK 5 — DAILY WORK LOG (continued)

Day 5 | 19 June 2026 | PDF Report Generator — Frontend Integration

Day 5 wired the frontend filter bar and handle picker to the new backend endpoints, completing the end-to-end report generation flow.

- Implemented the search flow in `frontend/js/nexus.js`: clicking the filter bar search button reads the From/To datetime inputs, calls `GET /api/nexus/handles-by-daterange`, and populates the handle picker with the matching results, showing a clear empty state when no handles have tweets in the selected range.
- Implemented checkbox selection state management using a Set of selected handles, with visual is-selected styling applied per row and a live-updating header count showing how many handles are currently selected out of the total found.
- Wired Select All and Clear actions in the picker to bulk-update the selection Set and re-render the list and Export button state in one pass.
- Implemented dynamic Export PDF button state — disabled with plain "Export PDF" label when no handles are selected, enabled and showing a live count "Export PDF (N)" once one or more handles are checked.
- Wired the Export action: on click, shows a toast notification ("Generating PDF for N handle(s)..."), POSTs the selected handles plus date range to `/api/nexus/report`, reads the binary PDF response, creates an object URL via Blob, and triggers a download using a dynamically created anchor element with a filename derived from the selected start date.
- Added error handling for the export flow — failed requests show an error toast ("PDF export failed") and log details to the console without leaving the UI in a broken state, restoring the Export button to its normal interactive state in all cases.

Day 6 | 20 June 2026 | Critical Bug Fix — SPA Router Intercepting File Downloads

During testing, a critical bug was discovered: clicking Export PDF did not download the file — instead the single-page-application router intercepted the click and attempted to navigate to it as if it were a page link.

- Diagnosed the root cause in `frontend/js/nav.js` `initPageTransitions()`: the global click handler that powers SPA-style navigation between pages listens for clicks on any anchor element and calls `preventDefault()` before doing a fetch-based page transition. The dynamically created download anchor (object URL pointing at the generated PDF Blob) was being caught by this same handler, so the browser never triggered the native download behaviour.
- Fixed the click handler to recognise and skip four additional categories of links that must never be intercepted: blob: URLs (used for in-memory file downloads created via `createObjectURL`), data: URIs, any anchor carrying the HTML download attribute, and reaffirmed the existing skip rules for hash links, external `http(s)` links, `javascript:` pseudo-links, and `mailto:` links.
- Verified the fix resolves the download for both the existing scraper page CSV export (which uses the same Blob + anchor pattern) and the new Nexus PDF report export, confirming the bug would have silently affected any future Blob-based downloads added to other pages as well.
- Re-tested the complete report flow after the fix: date range search, handle selection, PDF generation, and successful file download all confirmed working in a single pass without any SPA navigation interference.
- Updated `backend/server.py` CORS middleware to add `expose_headers` for `Content-Disposition` and `Content-Length` — required so the browser-side `fetch()` call in `nexus.js` can read the filename and size from the response headers when constructing the download.
- Added `reportlab` to `backend/requirements.txt` and the root `requirements.txt` to ensure the PDF generation dependency is captured for clean environment setup.

Day 7 | 21 June 2026 | End-to-End Testing & Documentation

The final day focused on comprehensive testing of the new report feature across multiple handle combinations and date ranges, plus documentation updates.

- Tested the date-range picker against multiple scenarios: no date bounds (returns all handles), start-only, end-only, and a fully bounded range, confirming the SQL query correctly handles each combination via the dynamic WHERE clause construction.
- Tested PDF generation with single-handle and multi-handle selections, confirming the cross-handle relationship sections (shared hashtags, cross mentions, shared links) correctly show "no data found" messaging when fewer than two handles share a signal, and correctly populate when overlapping patterns exist.
- Verified the auto-generated inference text reads correctly across different data shapes — including the case of a single selected handle, where cross-handle inferences gracefully report no relationships rather than producing malformed text.

- Confirmed the downloaded PDF opens correctly, paginates cleanly using KeepTogether blocks so individual handle profile sections never split awkwardly across a page break, and that all tables render with correct alternating row colours and grid lines.
- Performed a final regression pass across all 11 existing pages to confirm the nav.js click-handler fix did not affect any standard in-app navigation — all page transitions, including from Nexus back to Overview, Handles, Network, and other pages, continue to work via the SPA router exactly as before.
- Updated project documentation to describe the new Nexus report feature, the date-range filter endpoint, and the SPA navigation exclusion rules for future contributors adding file-download functionality to other pages.

WEEK 5 — MEMBER CONTRIBUTIONS

Rishav Pal — Data Engineer

- Led repository consolidation — initialised git version control, structured .gitignore, and committed the full production baseline at the start of the week.
- Built GET /api/nexus/handles-by-daterange in backend/routes/nexus.py with the parse_dt() flexible datetime parser supporting four date/time format variants and graceful fallback when bounds are omitted.
- Designed and implemented the per-handle data gathering pipeline for the PDF report endpoint — profile, date-bounded tweets, top hashtags, top mentions, top links, top topics, aggregate stats, and most-liked tweet per selected handle.
- Diagnosed and confirmed the root cause of the SPA router intercepting Blob-based file downloads, validating the fix against both the scraper CSV export and the new PDF report export to ensure no regression on either flow.
- Updated backend/requirements.txt and root requirements.txt with reportlab, and configured CORS expose_headers for Content-Disposition and Content-Length.
- Conducted backend regression testing of the date-range query across all bound combinations (none, start-only, end-only, full range).

Sayantana Mandal — OSINT Analyst

- Designed the cross-handle relationship detection logic for the PDF report: shared hashtag coordination signals, directional cross-mention pairs, and shared link domain analysis across an arbitrary selected handle set.
- Built the automated inference engine that converts raw relationship data into plain-English analytical findings — most active handle, most influential handle, strongest coordination signal, mention relationship summary, and common topics.
- Designed the full PDF report layout and content structure: cover section, per-handle profile sections with stats tables and most-liked-tweet quote blocks, cross-handle relationship tables, and the final inferences section.
- Styled the PDF document to match the platform's editorial design language using ReportLab — consistent typography tiers, black header bars, thin grid lines, and alternating row shading across every table.
- Conducted end-to-end QA of the report feature across single-handle and multi-handle selections, verifying correct messaging for zero-overlap relationship cases.
- Updated project documentation describing the Nexus report feature and date-range filter for future reference.

Shivam Kumar — Graph Analyst

- Designed and built the filter bar UI in frontend/nexus.html with From/To datetime-local inputs, primary search action, live status text, and Export PDF button matching the dark Nexus canvas aesthetic.
- Built the handle-picker dropdown component in frontend/js/nexus.js — checkbox list with per-handle metadata (display name, formatted follower count, tweet count in range), Select All / Clear actions, and live selection-count header.
- Implemented the full picker interaction logic: Set-based selection state, dynamic Export button label and enabled state, and the export trigger flow with toast notifications for in-progress and error states.
- Implemented the Blob-based PDF download flow — POST request, binary response handling, object URL creation, and anchor-triggered download with a date-derived filename.
- Fixed the critical bug in frontend/js/nav.js initPageTransitions() — extended the click-handler skip list to exclude blob: URLs, data: URIs, and download-attribute anchors, resolving the file download interception issue.
- Performed full regression testing across all 11 pages to confirm the nav.js fix preserved correct SPA routing behaviour for every existing in-app navigation path.

WEEK 5 — TECHNICAL SUMMARY

Feature Overview — Date-Range Report Generator

Week 5 delivered a complete intelligence reporting workflow on the Nexus page: a user narrows the dataset by date range, selects one or more handles from a live-populated picker showing tweet volume per handle, and exports a structured PDF containing individual handle profiles, cross-handle coordination signals, and automatically generated analytical findings — all without leaving the graph explorer.

Component	Technology	Detail
Date Range Query	PostgreSQL	Dynamic WHERE clause over tweets.timestamp, joined to profiles, grouped per handle
Datetime Parsing	Python (parse_dt)	4 format variants supported, UTC-aware, graceful fallback on omitted bounds
Handle Picker UI	Vanilla JS + CSS	Checkbox list, Select All/Clear, live count header, per-row tweet volume
Report Endpoint	FastAPI POST	/api/nexus/report — validates handles, gathers per-handle + cross-handle data
Relationship Engine	Python set/dict logic	Shared hashtags, cross mentions, shared links — scoped to selected handle set
Inference Engine	Python heuristics	Most active, most influential, coordination signal, mention summary, common topics
PDF Generation	ReportLab	SimpleDocTemplate, KeepTogether sections, styled tables matching platform design
Download Delivery	FastAPI Response	Full buffer in Response (not Streaming) to avoid async BytesIO GC issues
Frontend Download	Blob + anchor	fetch → blob → object URL → dynamically created anchor → programmatic click
Critical Fix	nav.js router	Excluded blob:, data:, and download-attribute anchors from SPA click interception

Week 5 Deliverables Summary

Deliverable	File	Status
Date-range handle query endpoint	backend/routes/nexus.py	✓ Complete
Flexible datetime parser	backend/routes/nexus.py	✓ Complete
PDF report generation endpoint	backend/routes/nexus.py	✓ Complete
Cross-handle relationship detection	backend/routes/nexus.py	✓ Complete
Automated inference engine	backend/routes/nexus.py	✓ Complete
ReportLab-styled PDF document	backend/routes/nexus.py	✓ Complete
Filter bar UI (From/To + status)	frontend/nexus.html	✓ Complete
Handle picker dropdown component	frontend/nexus.html + CSS	✓ Complete
Picker selection + export logic	frontend/js/nexus.js	✓ Complete
Blob-based PDF download flow	frontend/js/nexus.js	✓ Complete
CRITICAL FIX — SPA router intercepting downloads	frontend/js/nav.js	✓ Fixed
CORS expose_headers for download	backend/server.py	✓ Complete
reportlab dependency added	requirements.txt (both)	✓ Complete
Git repository initialised	project root	✓ Complete

PDF Report — Content Structure

The generated PDF report follows a fixed three-section structure regardless of how many handles are selected. Section 1 (Handle Profiles) repeats once per selected handle: a stats table (followers, following, tweets, total likes, total retweets,

average likes), side-by-side top-hashtag and top-mention tables, a topics summary line, and a quoted block showing the single most-liked tweet with its engagement numbers and timestamp. Section 2 (Cross-Handle Relationships) presents three tables — shared hashtags with the handles using each one, directional cross-handle mentions, and shared links grouped by domain. Section 3 (Inferences & Analysis) lists the automatically generated findings as a numbered list in plain English, giving a non-technical reader an immediate summary of the most important patterns in the selected data without needing to interpret the raw tables.